上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU

HBase和NoSQL

丁玥

上海交通大学计算机学院

dingyue@sjtu.edu.cn

2026年秋季

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Hadoop HDFS的局限性

2 HBase数据模型

3 HBase运行机制

4 NoSQL

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



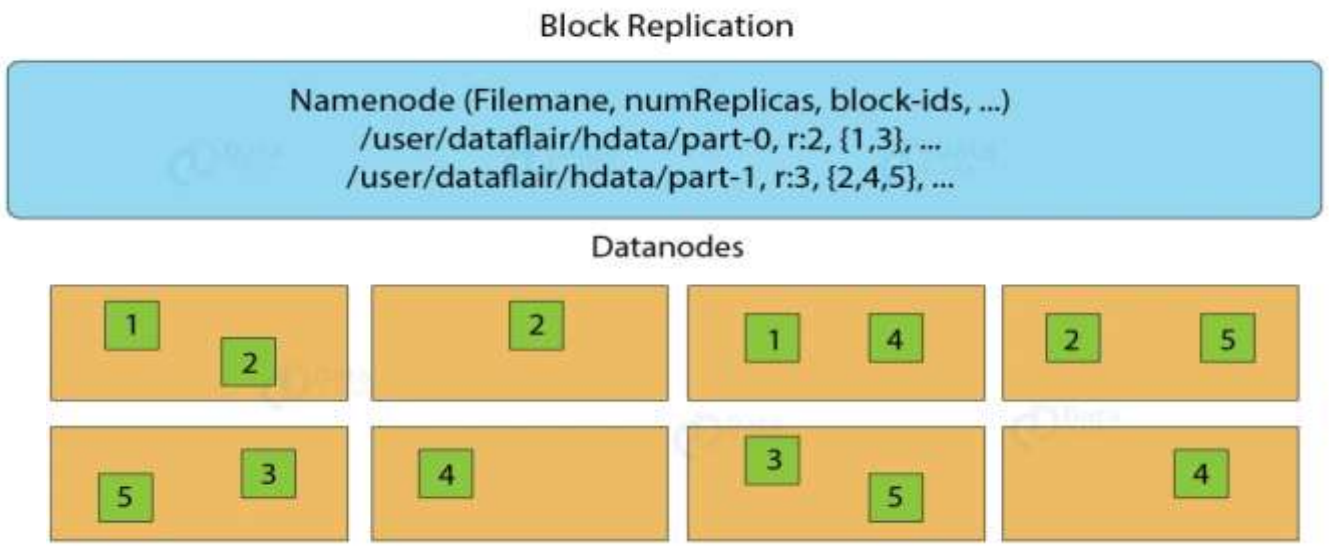
1 Hadoop HDFS的局限性

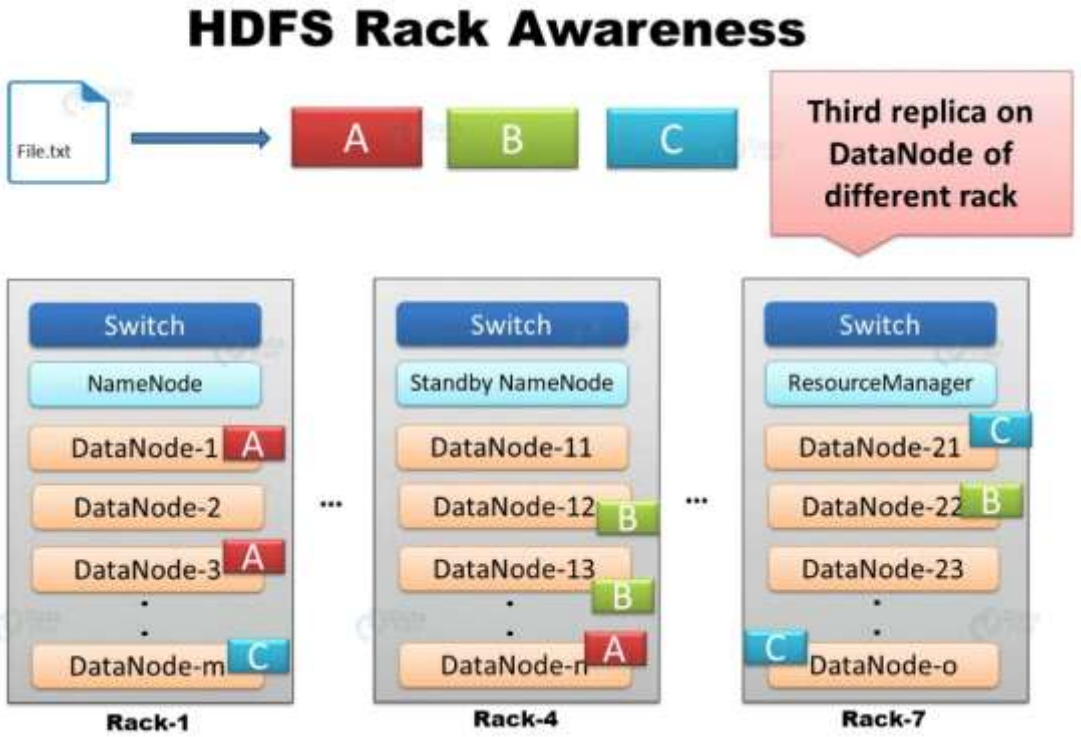
2 HBase数据模型

3 HBase运行机制

4 NoSQL

Hadoop HDFS主要解决大数据“如何存”的问题

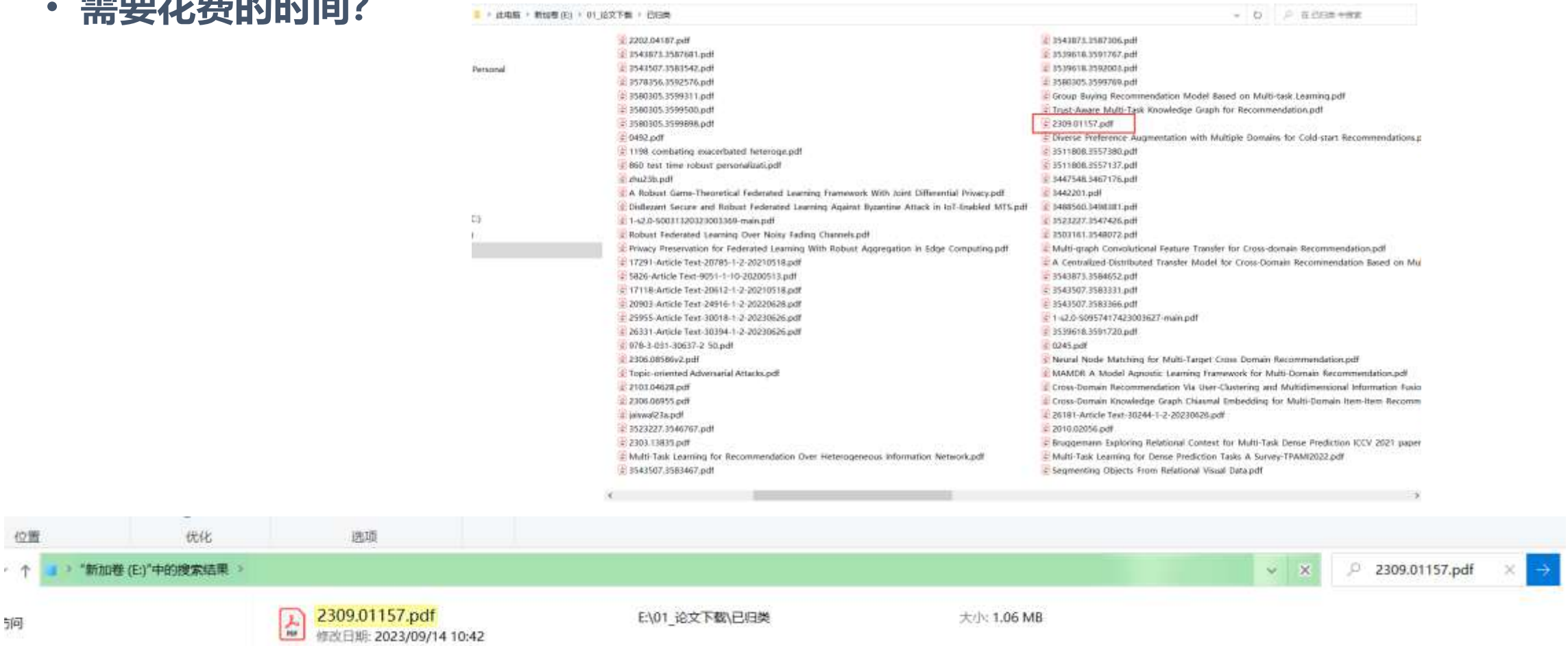




Hadoop HDFS的局限性

考虑一个简单的问题，windows下如何快速查找想要的文件

- 需要花费的时间?



Hadoop HDFS的局限性

那么如何能够提升查询速度?

- 建立索引是最简单直接的办法





Q: 单机操作系统比如windows如何建立索引



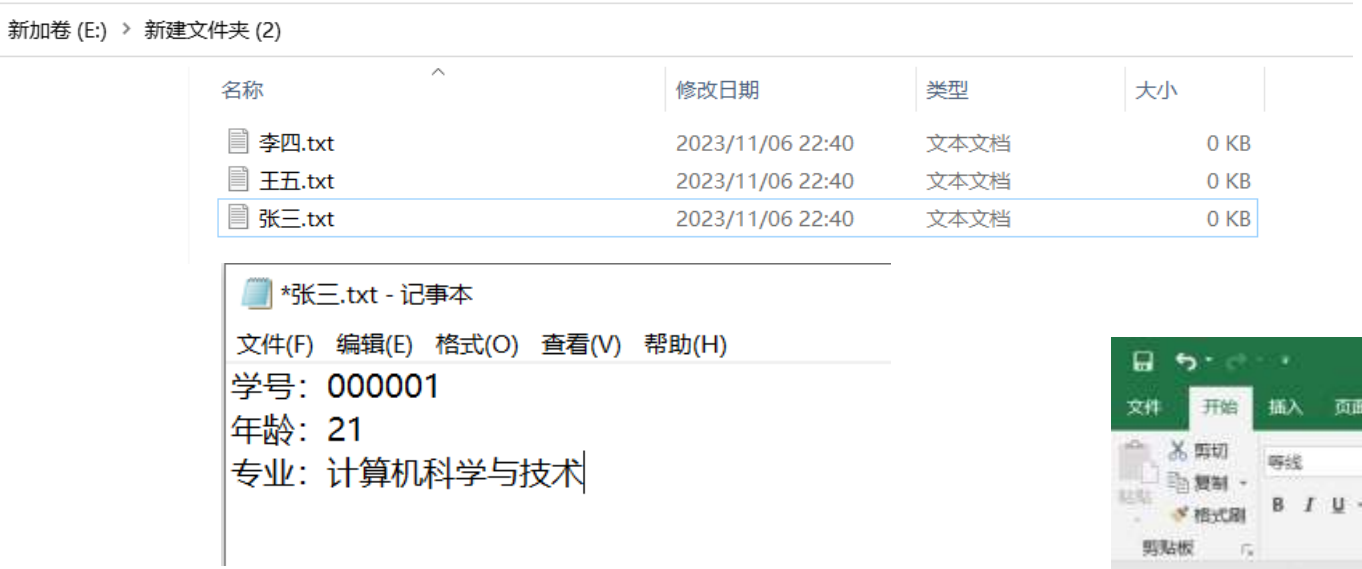


Hadoop HDFS的局限性

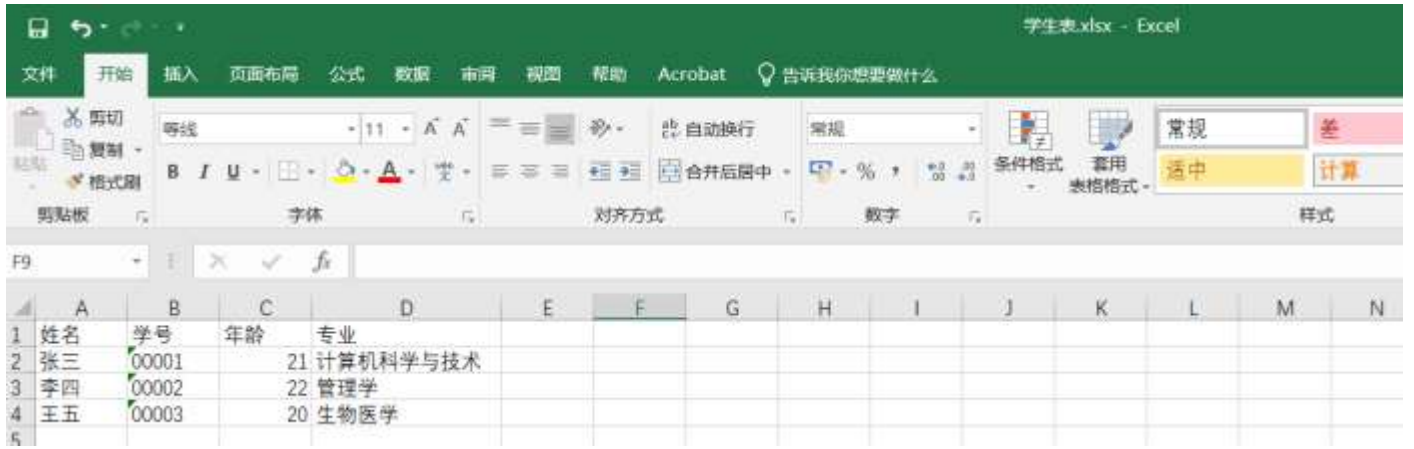
如果我们碰到的是这样的问题，我们的数据该如何存储，使用HDFS合适吗？

- Case: 学生信息的存储，需要用于查询成绩，下面两个方案，你会选择哪一种方案？

方案一



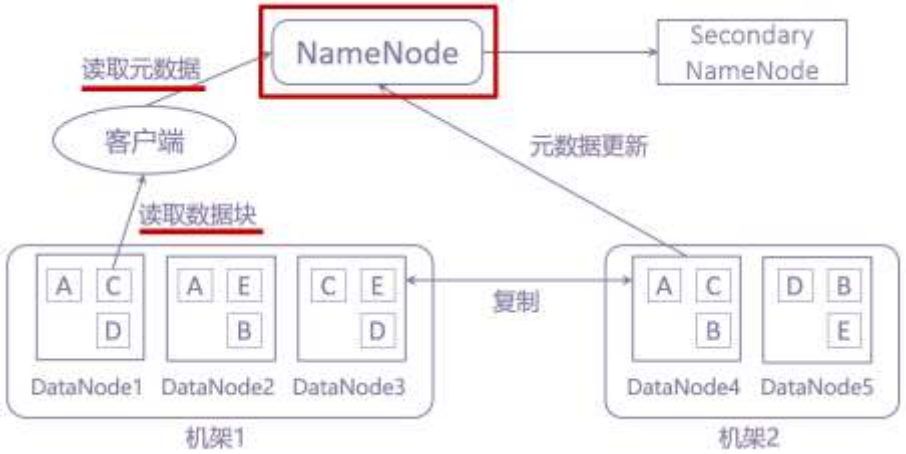
方案二

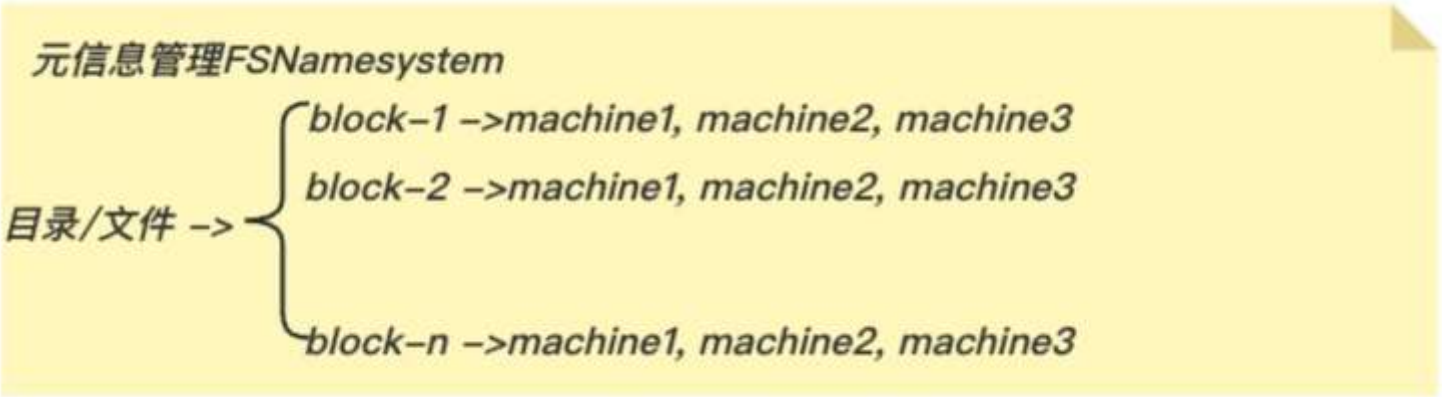


Hadoop HDFS的索引问题

Hadoop HDFS默认以顺序方式访问数据

- 什么情况适合建立索引？文件比较多但每次查询都不知道文件的路径时
- **HDFS 中的每个文件都要在NameNode上建立一个索引，当文件很多时会产生很多的索引记录**
  - 会大量占用NameNode的内存空间
  - 索引文件过大会使得索引速度变慢

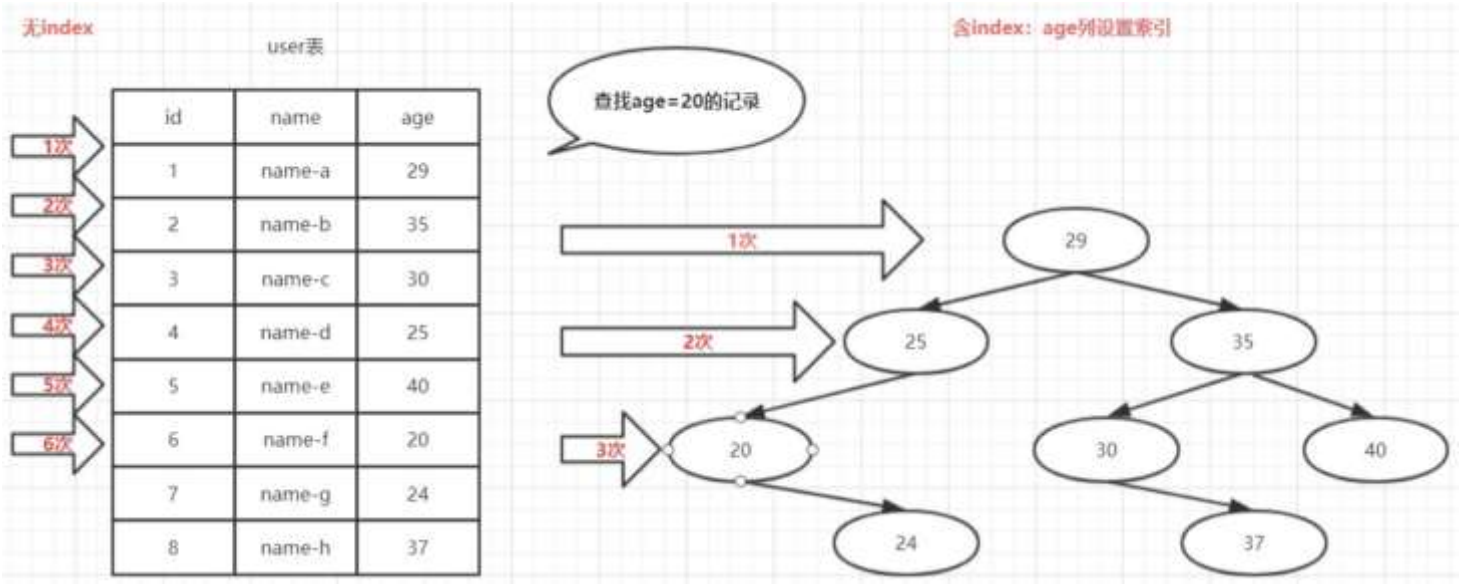




- **在分布式系统中，如果需要处理的是大量结构化（表格）数据，需要一种比HDFS更好的方案**

Hadoop HDFS的局限性

- 对于结构化（表格）数据，我们可以通过关系型数据库来实现存储和读取
  - 关系型数据库能否非常灵活地实现随机访问
    - 索引
    - 通用的SQL语句



数据库索引的例子

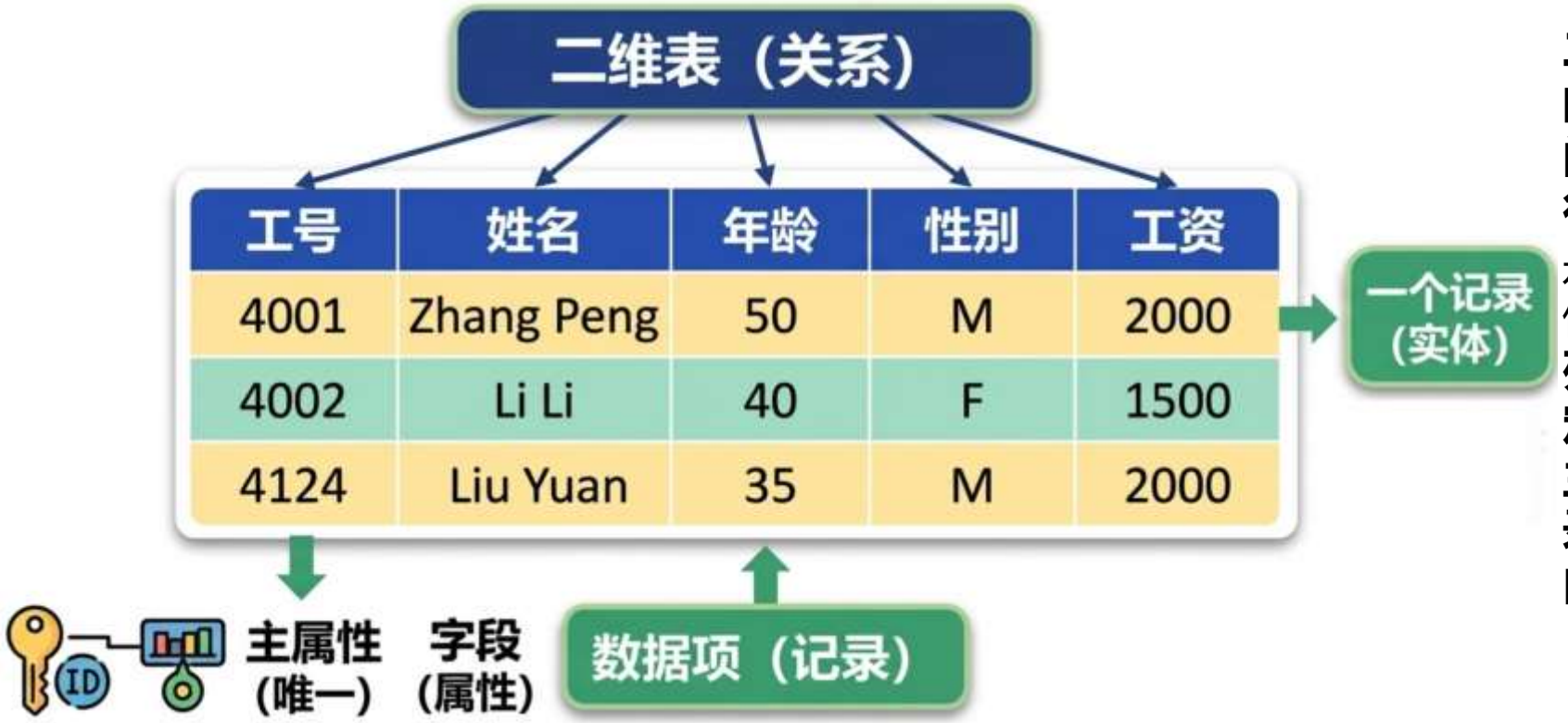
SQL语句查询

```
-- Read
SELECT * FROM inventory;
```

student_numb...	student_name	dept_name	subject	marks	rank_num	dense_rank_num
9	Alicia	English	History	80	1	1
10	Sofia	English	History	70	2	2
11	Sofia	English	Grammar	70	2	2
8	John	English	Literature	75	4	3
4	Fred	CS	Programming	80	5	4
7	Jason	CS	Programming	80	5	4
1	Tom	CS	Programming	90	7	5
3	Rob	CS	Programming	95	8	6
6	Jason	CS	Database	98	9	7
2	Tom	CS	Database	100	10	8
5	Alter	CS	Database	100	10	8

关系型数据库以行和列的形式存储数据，这一系列的行和列被称为表，一组表组成了数据库。

- 例：员工信息表



说明：

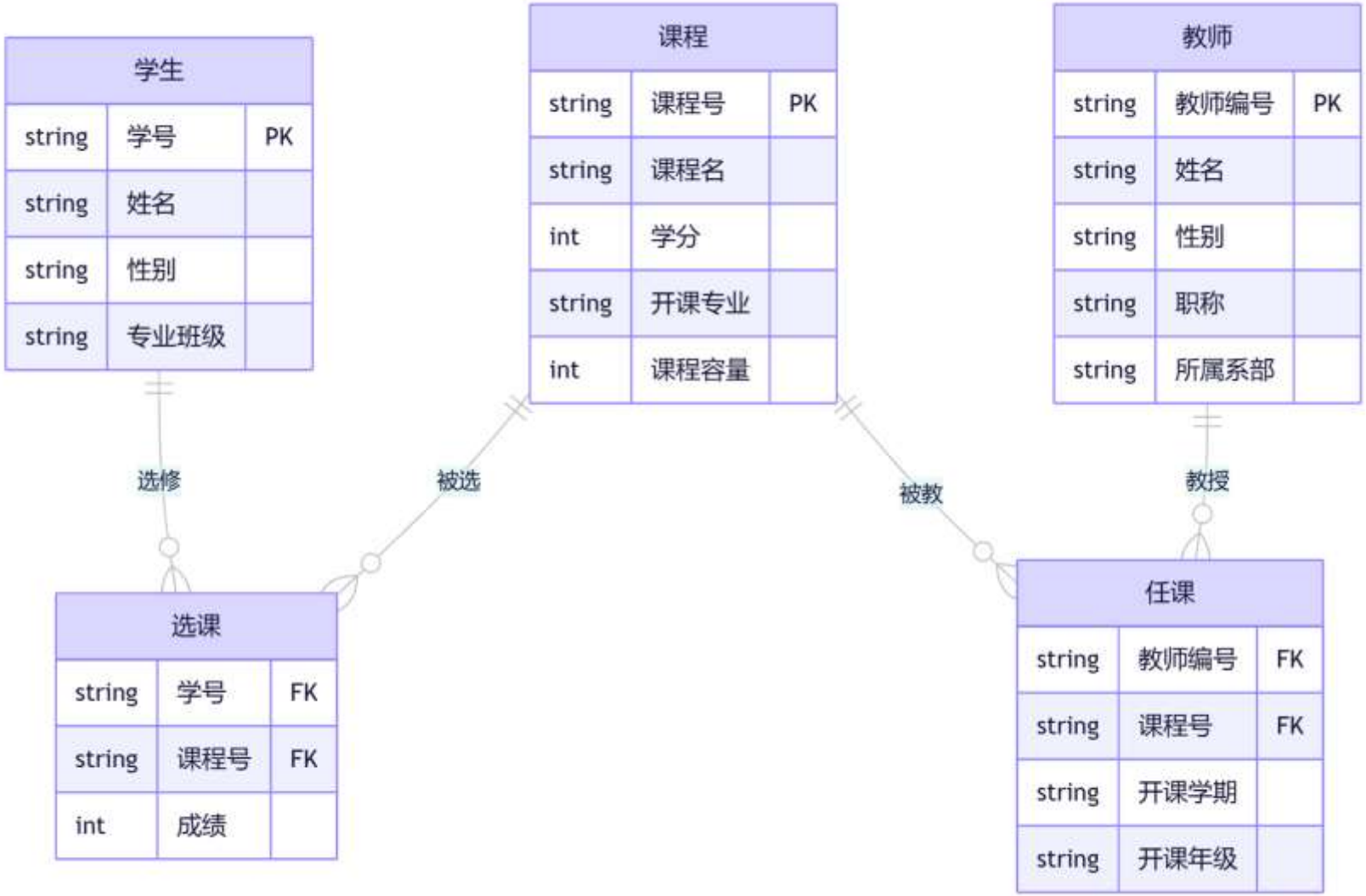
二维表(table)：由纵向的列和横向的行组成，描述的是存储数据对象之间的关系

行(row)：也叫**元组或记录**，在表中是一条横向的数据集合，代表一个实体（entity）。

列(column)：也叫**字段或属性**，**列定义了表中的数据结构。**

主属性(主键)：**值唯一地标识一个记录。主属性可以是一个属性，也可以由多个属性共同组成。**

- **在关系型数据库中，实体以及实体间的联系均由单一的结构类型来表示，这种逻辑结构是一张二维表**
  - 例：学生选课系统数据库逻辑结构



- **关系型数据库的核心是其结构化的查询语言（Structured Query Language, SQL）**
  - SQL 涵盖了数据的**查询、操纵、定义和控制**，是通用且简单易懂的数据库管理语言
  - **数据查询语言的基本语法结构**

```
1. SELECT FROM 表名1,表2
2. where 查询条件 #可以组合 and、or、 not、 =、between、and、in、like 等;
3. group by 分组字段
4. having（分组后的过滤条件）
5. order by 排序字段和规则;
```

简单的SQL查询语句示例

查询结果

SQLQuery2.sql -...MAXSU\hema (58))* | SQLQuery1.sql -...MAXSU\hem

```
SELECT
    *
FROM
    sales.customers
WHERE
    state = 'CA'
ORDER BY
    first_name;
```

100 %

结果 消息

	customer_id	first_name	last_name	phone	email	street
1	673	Adam	Henderson	NULL	adam.henderson@hotmail.com	167 James St.
2	1261	Adelaida	Hancock	NULL	adelaida.hancock@aol.com	669 S. Gartner Street
3	574	Adriene	Rivera	NULL	adriene.rivera@hotmail.com	973 Yukon Avenue
4	1353	Agatha	Daniels	NULL	agatha.daniels@yahoo.com	125 Canal St.
5	735	Aide	Franco	NULL	aide.franco@msn.com	8017 Lake Forest St.
6	952	Aileen	Marquez	NULL	aileen.marquez@msn.com	8899 Newbridge Street
7	697	Alane	Mccarty	(619) 377-8586	alane.mccarty@hotmail.com	8254 Hilldale Street

原始数据

customer_id	first_name	last_name	phone	email	street	city	state	zip_code
1	Debra	Burks	NULL	debra.burks@yahoo.com	9273 Thome Ave.	Orchard Park	NY	14127
2	Kasha	Todd	NULL	kasha.todd@yahoo.com	910 Vine Street	Campbell	CA	95008
3	Tameka	Fisher	NULL	tameka.fisher@aol.com	769C Honey Creek St.	Redondo Beach	CA	90278
4	Daryl	Spence	NULL	daryl.spence@aol.com	988 Pearl Lane	Uniondale	NY	11553
5	Charolette	Rice	(916) 381-6003	charolette.rice@msn.com	107 River Dr.	Sacramento	CA	95820
6	Lyndsey	Bean	NULL	lyndsey.bean@hotmail.com	769 West Road	Fairport	NY	14450
7	Latasha	Hays	(716) 986-3359	latasha.hays@hotmail.com	7014 Manor Station Rd.	Buffalo	NY	14215
8	Jacquline	Duncan	NULL	jacquline.duncan@yahoo.com	15 Brown St.	Jackson Heights	NY	11372
9	Genoveva	Baldwin	NULL	genoveva.baldwin@msn.com	8550 Spruce Drive	Port Washington	NY	11050
10	Pamelia	Newman	NULL	pamelia.newman@gmail.com	476 Chestnut Ave.	Monroe	NY	10950

关系型数据库的优点

- 容易理解：二维表结构**非常贴近逻辑世界的概念**
- 使用方便：**通用的 SQL** 使用户操作关系型数据库非常方便
- 易于维护：丰富的完整性**大大减少了数据冗余和数据不一致的问题**。关系型数据库提供对**事务的支持**，能保证系统中事务的正确执行，同时**提供事务的恢复、回滚、并发控制和死锁问题的解决**。

关系型数据库的缺点

- **高并发读写能力差**：**数据库的最大连接数有限（MySQL默认100，最大16384），硬盘 I/O 有限**，不能满足很多人同时连接，而互联网时代网站类用户的**并发性访问非常高**
- **对海量数据的读写效率低**：若表中数据量太大，则每次的读写速度将非常缓慢
- **扩展性差**：
  - 在一般的关系型数据库系统中，通过**升级数据库服务器的硬件配置**的纵向扩展方式**终会达到硬件性能的瓶颈，无法应对互联网数据爆炸式增长的需求**
  - **横向扩展**采用多台计算机组成集群共同完成对数据的存储、管理和处理，集群对数据进行分散存储和统一管理，可满足对海量数据的存储和处理的需求。但是由于**关系型数据库具有数据模型、完整性约束和事务的强一致性等特点，导致其难以实现高效率的、易横向扩展的分布式架构。**

关系型数据库

- **我们需要一种新的技术来解决海量数据的查询问题，解决传统单机关系型数据库的可扩展性和性能问题**
  - Hadoop解决“如何存”的问题，但是解决不了“如何查”。Hadoop**只能执行批量处理**，并且**只以顺序方式访问数据**，这意味每次要搜索整个数据集，无法实现随机访问
  - **传统单机关系型数据库，其主要特点是方便随机访问，但它们却不能用于海量数据的存储**

我们需要在大数据下能够实现灵活访问、具有高可扩展、 低写入/查询延迟的系统

- **键值存储数据库：**使用简单的键值方法来存储数据。键值数据库将数据存储为键值对集合，其中键作为唯一标识符。代表软件：redis
- **面向文档数据库：**此类数据库可存放并获取文档，可以是XML、JSON、BSON等格式，这些文档具备可描述性（self-describing），呈现分层的树状结构（hierarchical tree data structure），可以包含映射表、集合和纯量值。代表软件：mongoDB
- **列存储数据库**：列式存储（column-based）是相对于传统关系型数据库的行式存储（Row-based storage）来说的。简单来说，两者的区别就是对表中数据的存储形式的差异。代表软件：APACHE HBASE

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Hadoop HDFS的局限性

2 HBase数据模型

3 HBase运行机制

4 NoSQL

HBase与传统关系数据库的对比分析

- 数据类型：关系数据库采用关系模型，**HBase则采用了更加简单的数据模型（非关系型）**
- 数据操作：关系数据库中包含了丰富的操作，其中会涉及复杂的多表连接。**HBase在设计上就避免了复杂的表和表之间的关系，只有简单的插入、查询、删除、清空等**
- 存储模式：**关系数据库是基于行模式存储的，HBase是基于列存储的**，每个列族都由几个文件保存，不同列族的文件是分离的
  - 用户在表中存储数据，每一行都有一个可排序的行键和任意多的列
  - 表在水平方向由一个或者多个列族组成，一个列族中可以包含任意多个列，**同一个列族里面的数据存储在一起**

HBase与传统关系数据库的对比分析

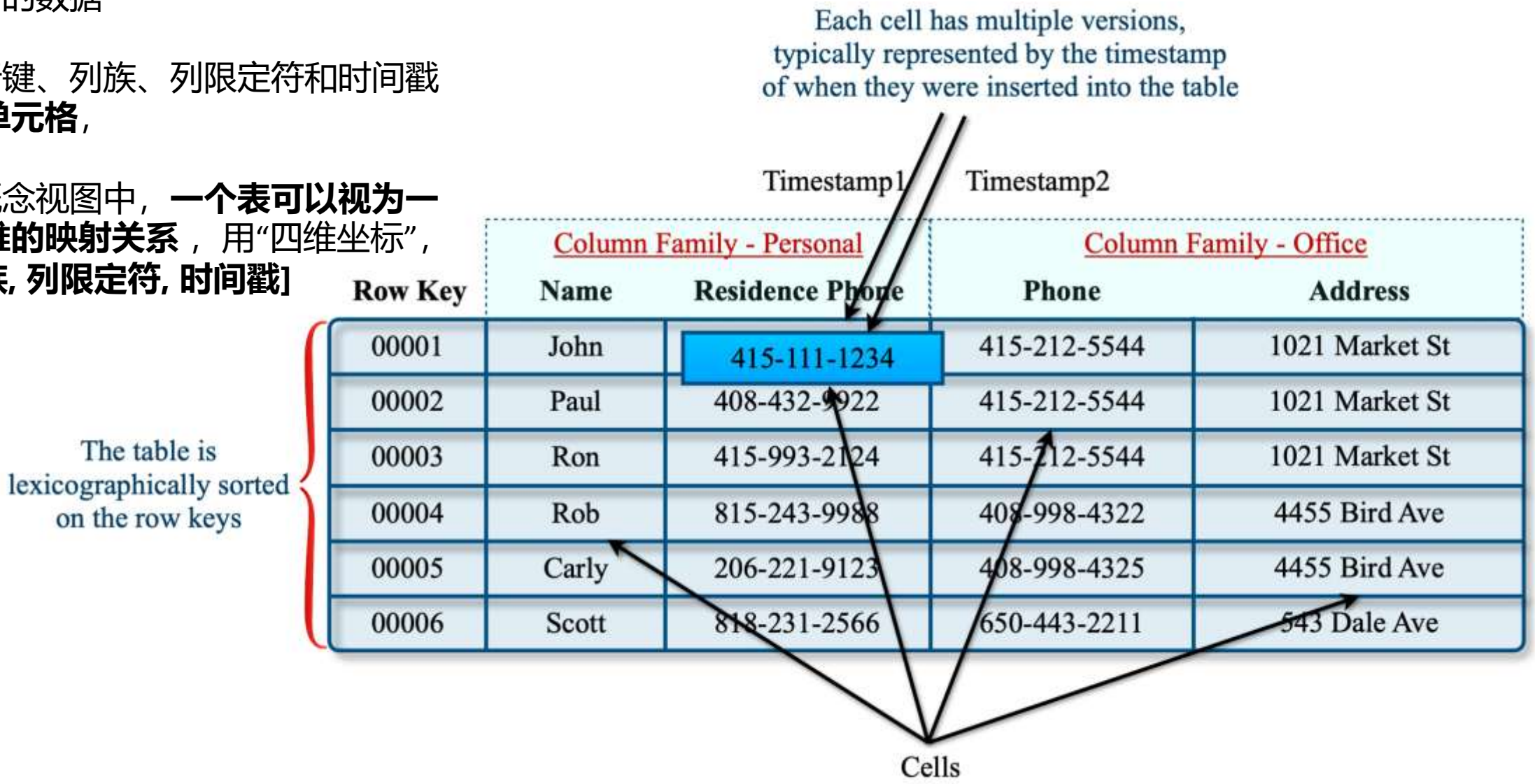
- 数据索引：关系数据库通常可以针对不同列构建复杂的多个索引，以提高数据访问性能。而HBase的**索引是行键、列族、列限定符和时间戳。HBase中的所有访问方法，或者通过行键访问，或者通过行键扫描，从而使得整个系统不会慢下来**
- 数据维护：在关系数据库中，更新操作会用最新的当前值去替换记录中原来的旧值，旧值被覆盖后就不会存在。而在HBase中执行更新操作时，**并不会删除数据旧的版本，而是生成一个新的版本，旧有的版本仍然保留**（这是和HDFS只允许追加不允许修改的特性相关的）
- 可伸缩性：关系数据库很难实现横向扩展，纵向扩展的空间也比较有限。相反，HBase**能够轻易地通过在集群中增加或者减少硬件数量来实现性能的伸缩**
  - HBase列族支持动态扩展，可以很轻松地添加一个列族或列，无需预先定义列的数量以及类型

HBase的数据概念视图

概念视图即**逻辑视图**，即如何理解HBase和操作HBase中的数据

HBase根据行键、列族、列限定符和时间戳来**确定一个单元格**，

在HBase的概念视图中，**一个表可以视为一个稀疏、多维的映射关系**，用“四维坐标”，即**[行键, 列族, 列限定符, 时间戳]**



HBase的物理视图

- 从概念视图层面，HBase中的每个表是许多行组成的，但是在**物理存储层面，它采用基于列的存储方式，而不是像传统关系数据库那样采用基于行的存储方式，这也是HBase和传统关系数据库的重要区别。**

概念视图

行键	时间戳	列族contents	列族anchor
"com.cnn.www"	t5		anchor:cnnsi.com="CNN"
	t4		anchor:my.look.ca="CNN.com"
	t3	contents:html="<html>..."	
	t2	contents:html="<html>..."	
	t1	contents:html="<html>..."	

概念视图中的空白单元格不会被存储，如果请求返回这些单元格，则会返回null

物理存储时，会存成两个小片段

列族contents

行键	时间戳	列族contents
"com.cnn.www"	t3	contents:html="<html>..."
	t2	contents:html="<html>..."
	t1	contents:html="<html>..."

列族anchor

行键	时间戳	列族anchor
"com.cnn.www"	t5	anchor:cnnsi.com="CNN"
	t4	anchor:my.look.ca="CNN.com"

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Hadoop HDFS的局限性

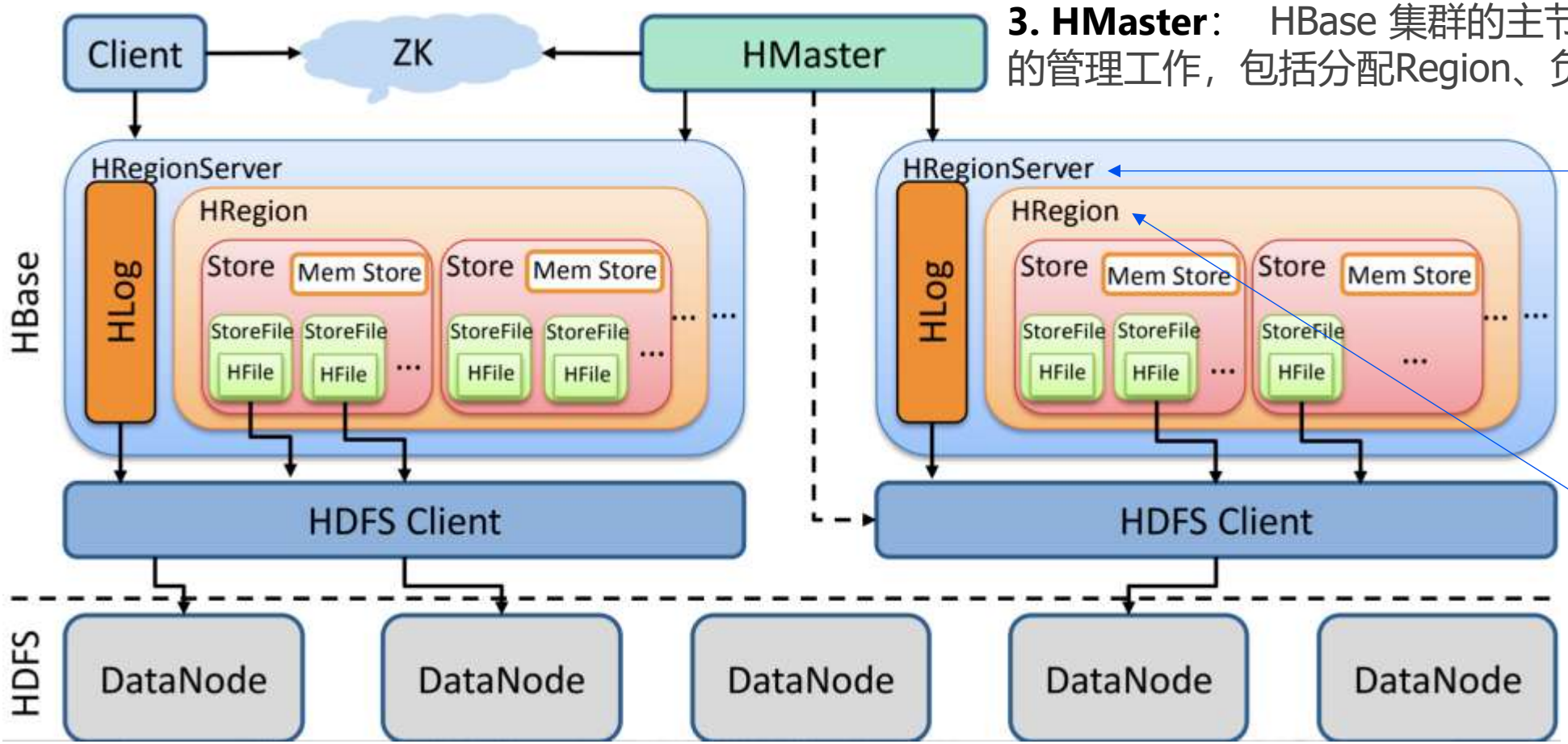
2 HBase数据模型

3 HBase运行机制

4 NoSQL

HBase的核心架构

4. ZooKeeper：用于分布式协调，管理集群的状态信息，如RegionServer的注册、故障检测等客户端并不依赖HMaster，而是通过Zookeeper获得Region位置信息，大多数客户端甚至从来不和HMaster通信，这种设计方式使得Master负载很小



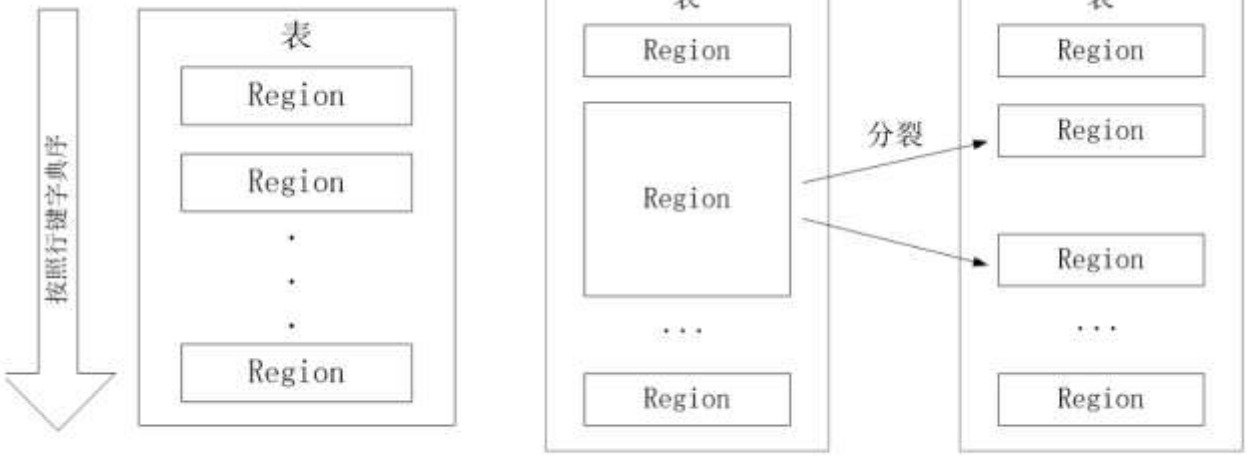
3. HMaster： HBase 集群的主节点，负责整个集群的管理工作，包括分配Region、负载均衡、维护数据

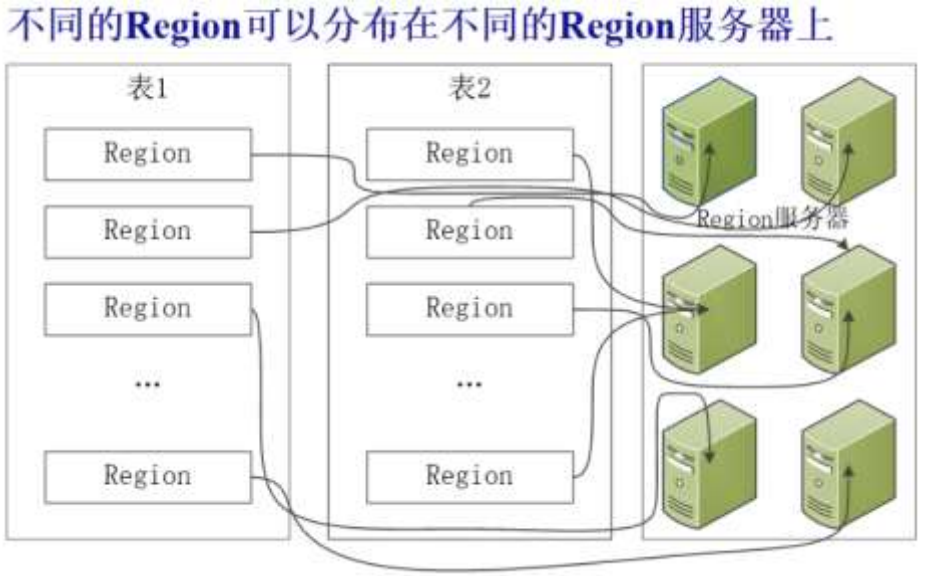
2. RegionServer：负责实际的数据存储和服务请求，每个RegionServer管理多个Region

1. Region：表被水平分割成多个Region，每个Region存储表的一部分数据

表和Region

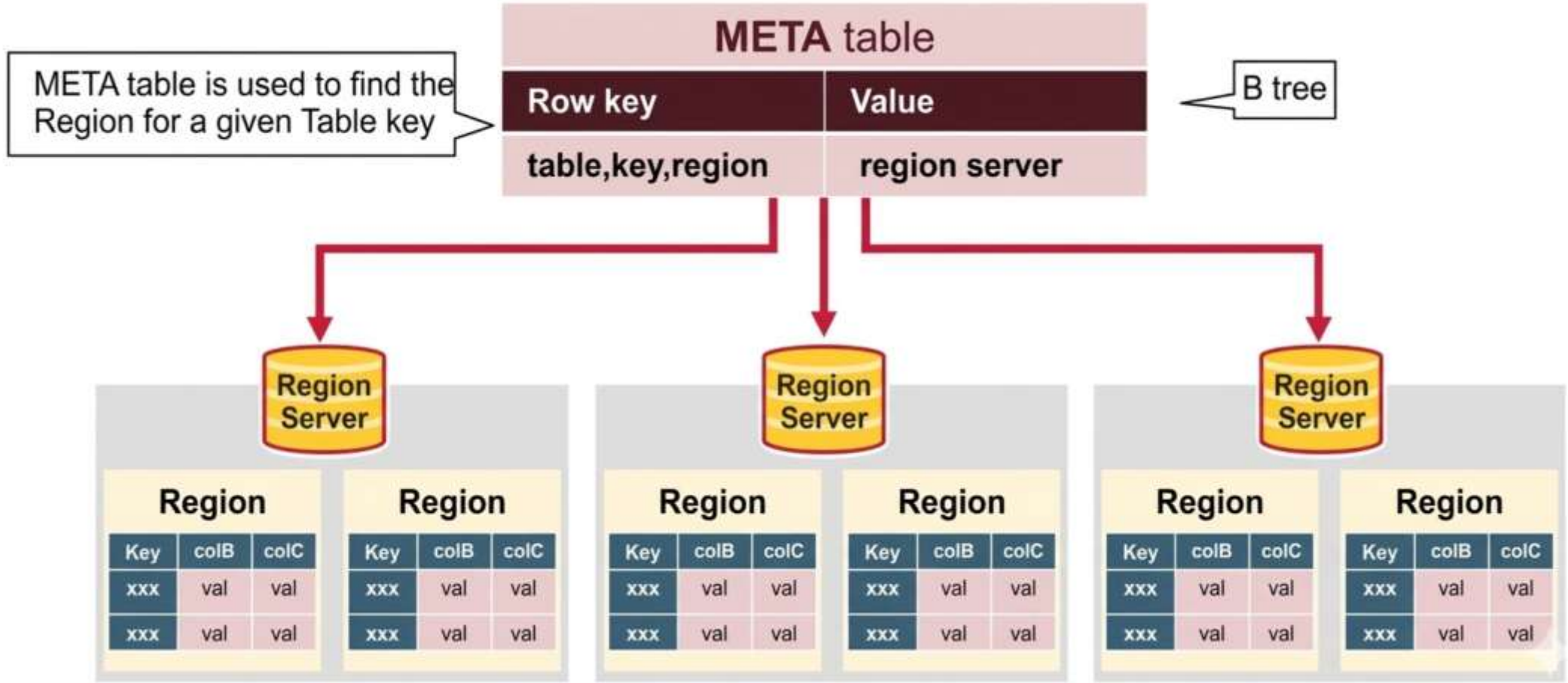
- Hbase数据库可以存储很多表，表中的行是根据行键值的字典序进行维护的，表中包含的行的数量可能非常庞大，无法存储在一台机器上，**需要分布存储到多台机器上**
- 因此需要根据行键的值对表中的行进行分区。**每个行区间构成一个分区，被称为Region**
- Region包含了位于某个值域区间内的所有数据，是负载均衡和数据分发的基本单位





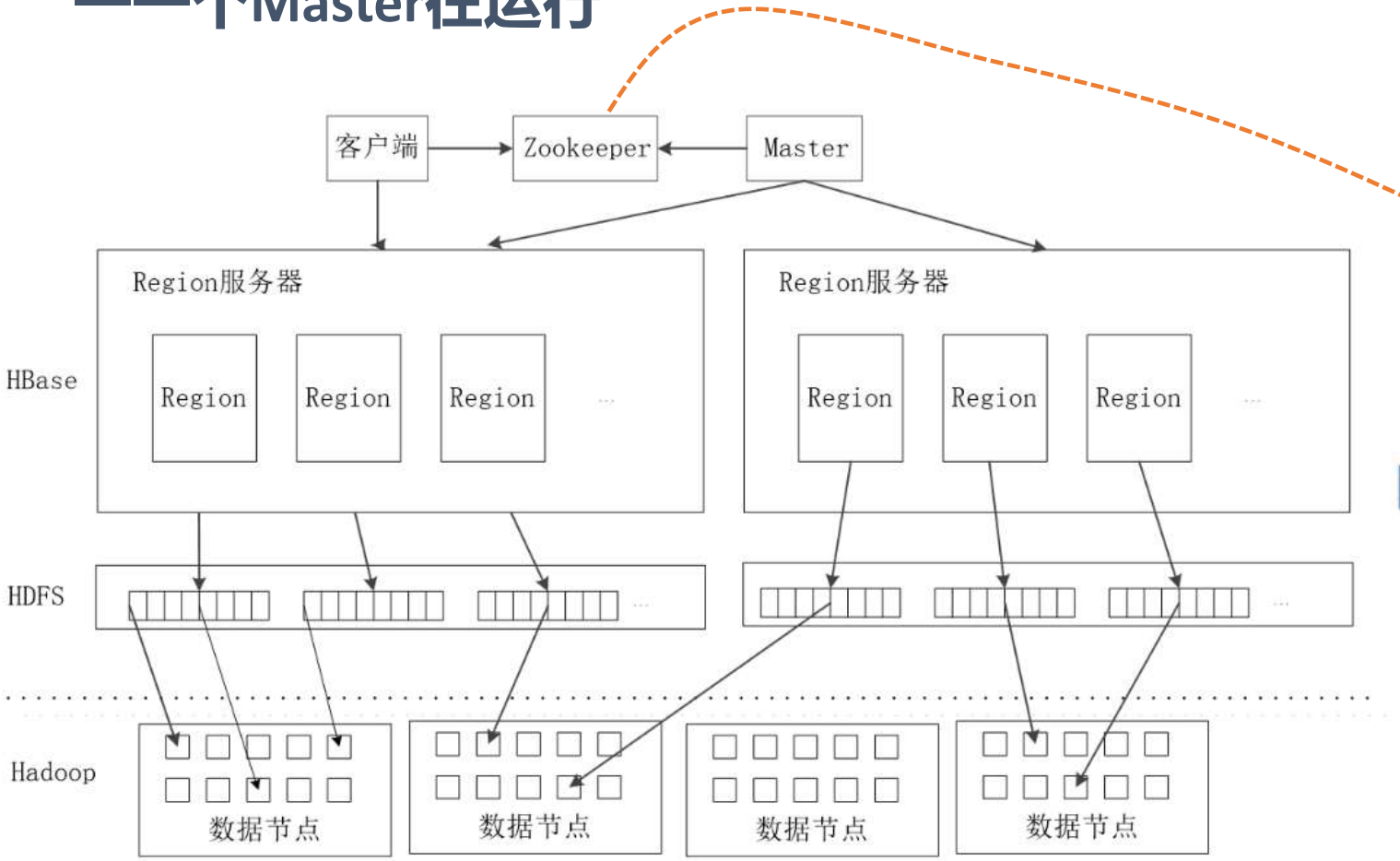
HBase的核心架构

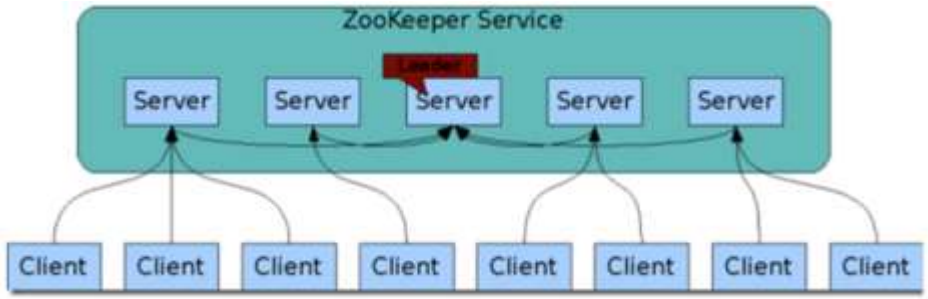
Region的定位: 每个Region都有一个RegionID来标识它的唯一性，一个Region标识符可以表示成**表名 + 开始主键+RegionID。**为了定位每个Region所在的位置，可以构建一张映射表即**“元数据表”(META表)**，每行包含两项内容：Region和Region服务器之间的对应关系。



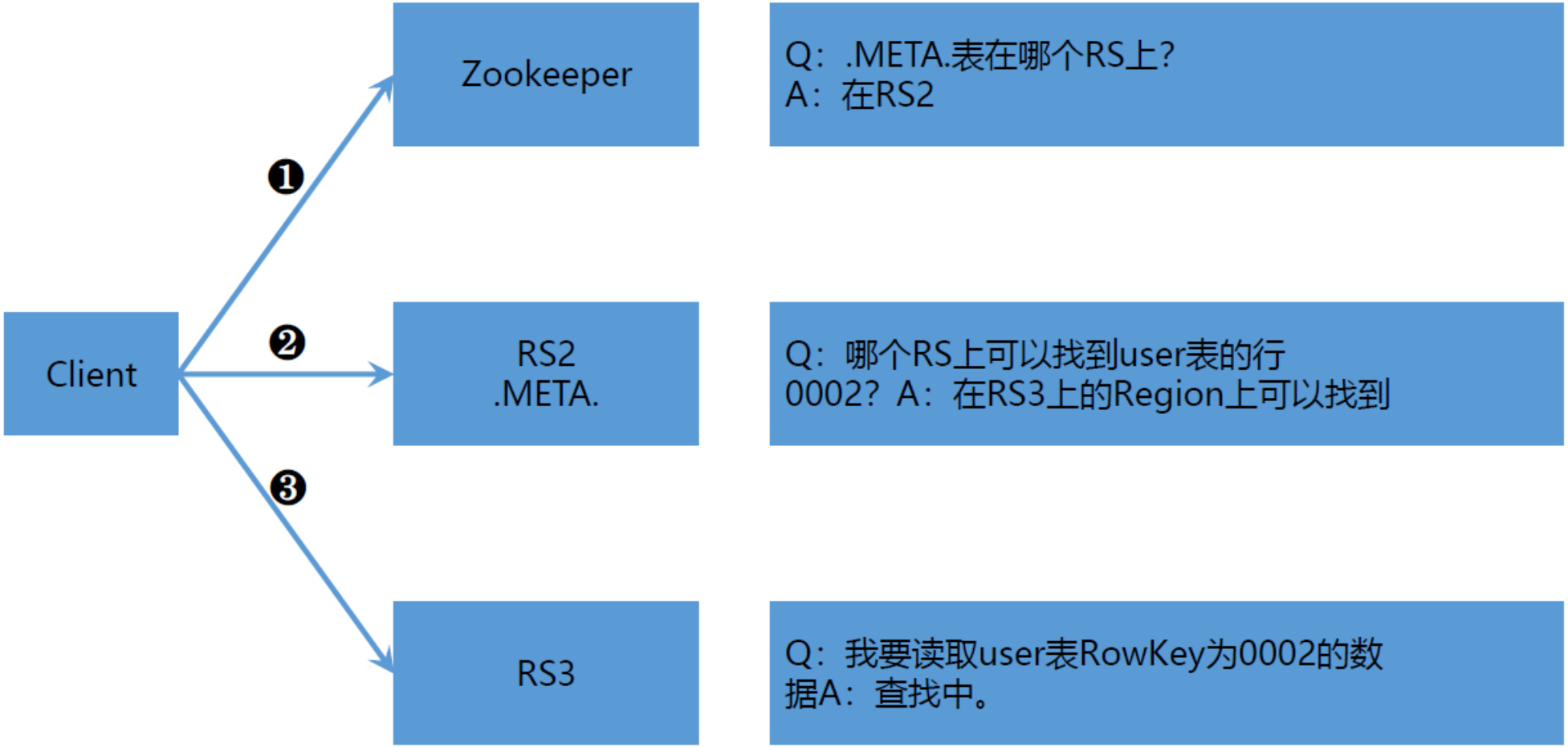
HBase系统架构

- Zookeeper可以帮助选举出一个HMaster作为集群的总管，并保证在任何时刻总有唯一一个Master在运行





HBase Region查找



RowKey的设计原则

RowKey的作用

- **唯一的标识一行记录，在增删改查的过程中RowKey就充当了主键的作用**
- 读写数据时通过 RowKey 找到对应的 Region

在HBase中检索数据时使用到RowKey的一共有三种方式：

- get：通过指定单个RowKey来获取对应的唯一一条记录；
- scan：通过设置StartRow和StopRow参数进行范围匹配（注意：如果不设置就是全表扫描）
- 模糊查询

全局有序

RowKey	personal			office	
	name	city	phone	tel	address
Row1	张三	上海	13111111111	010-1111111	帝都大厦-18F-01
Row11	李四	上海		010-4444444	帝都大厦-19F-02
Row2	王五	武汉	18655555555	010-3333333	帝都大厦-18F-02
Row3	赵六		15166666666		帝都大厦-18F-03
Row4	孙七	北京		010-7777777	帝都大厦-18F-04
Row5	周八	深圳	15388888888		帝都大厦-18F-05
Row6	吴九	杭州		010-9999999	帝都大厦-18F-06
Row7	郑十	武汉	13599999999	010-5555555	帝都大厦-18F-07

RowKey的设计原则

- **HBase的每个region都有StartRowKey和StopRowKey**，用来表示 Region存储的RowKey的范围，HBase表里面的数据按照RowKey来分散存储到不同的Region里
- **热点现象**：当大量请求访问HBase集群的一个或少数几个节点，造成少数RegionServer的读写请求过多，负载过大，而其他RegionServer负载却很小，这样就造成**热点现象**

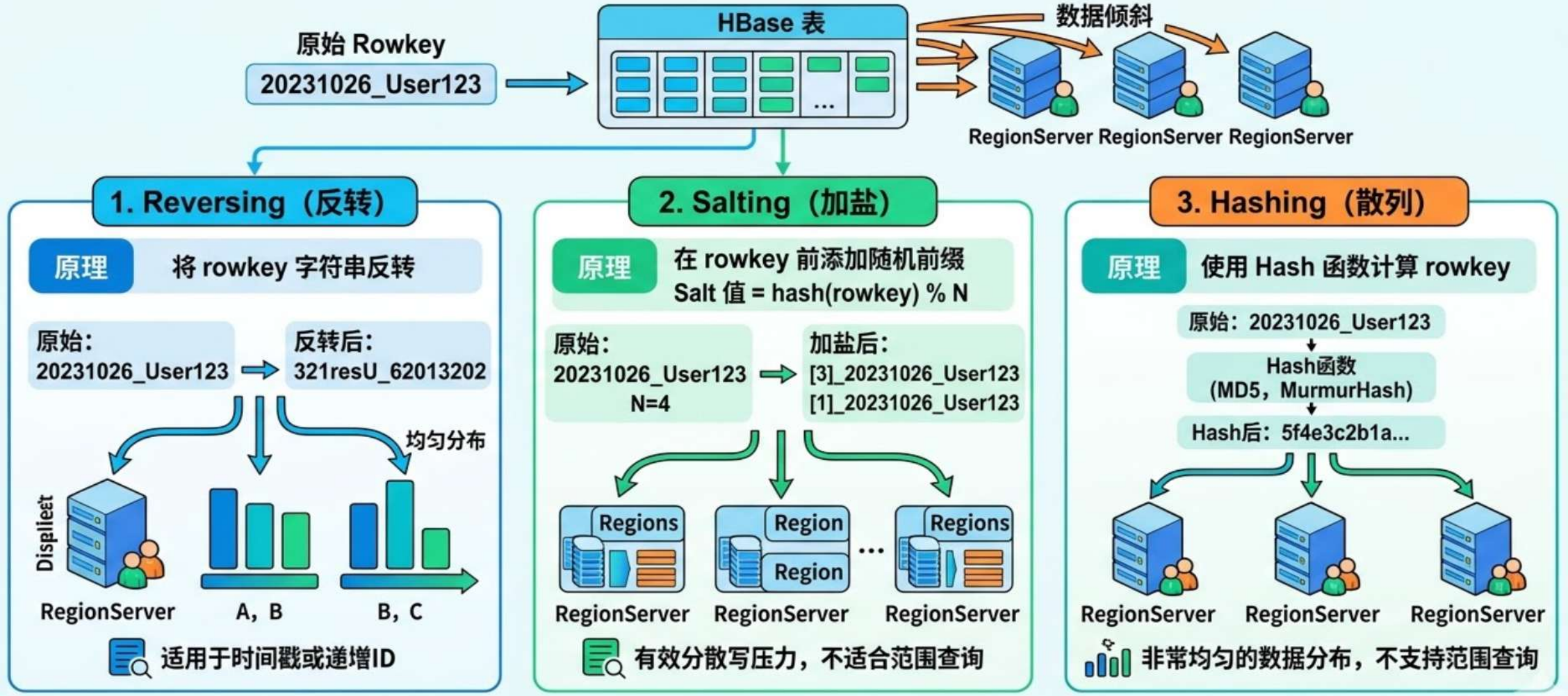
RowKey的设计原则

- 避免热点现象：结合业务的特点，并考虑高频查询，尽可能的将数据打散到整个集群



- RowKey设计的三种方式：反转（Reversing），加盐(Salting)和散列(Hashing)

RowKey的设计原则



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Hadoop HDFS的局限性

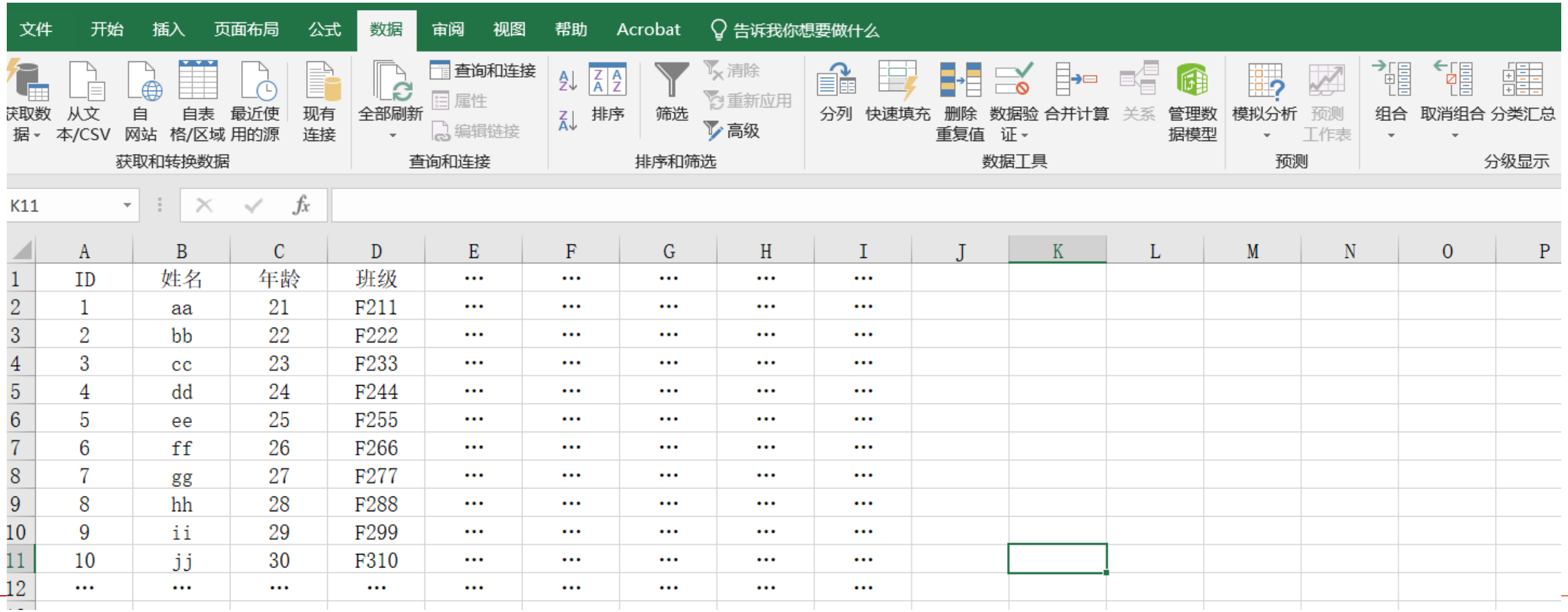
2 HBase数据模型

3 HBase运行机制

4 NoSQL

问题：如何快速从大量的结构化（表格）数据中找到并返回想要的结果?

- 例：从excel表格上找到年龄>22的学生



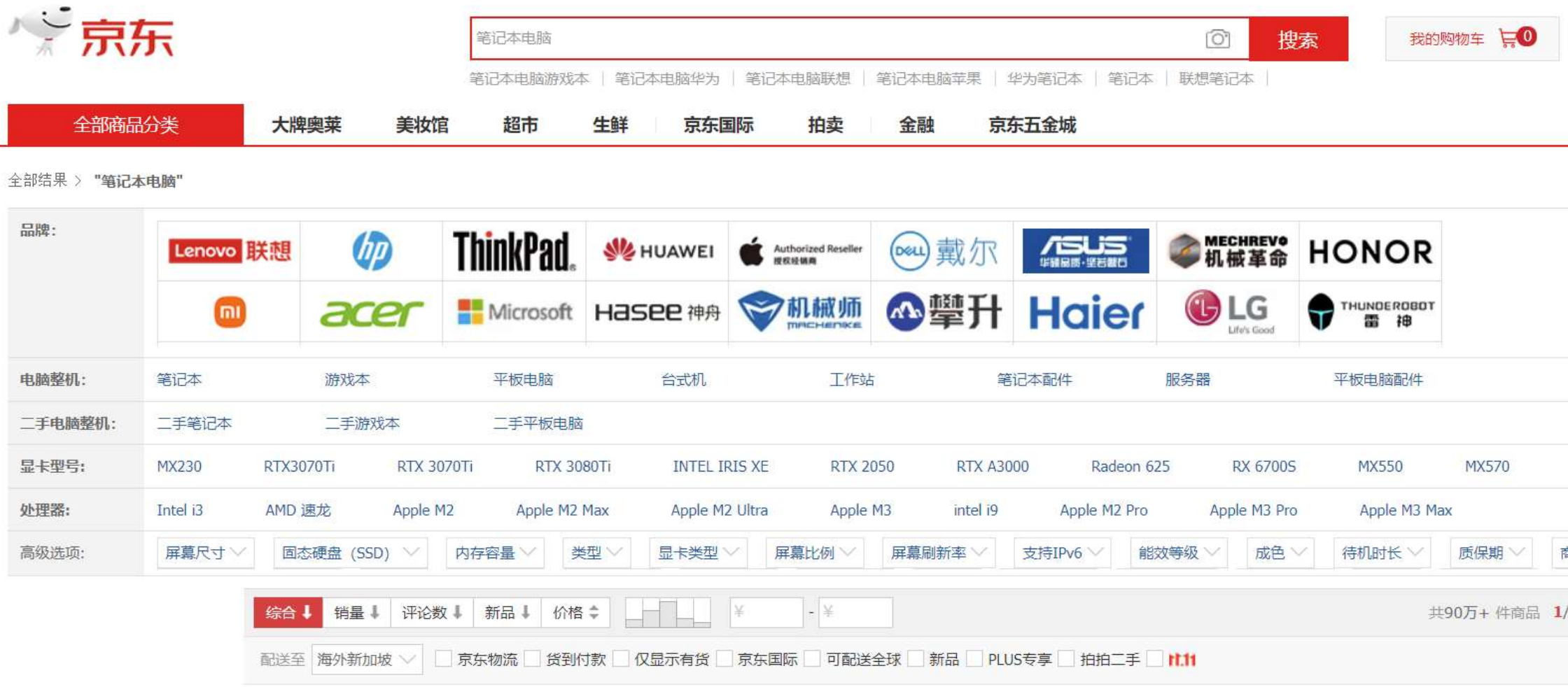
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	ID	姓名	年龄	班级	…	…	…	…	…							
2	1	aa	21	F211	…	…	…	…	…							
3	2	bb	22	F222	…	…	…	…	…							
4	3	cc	23	F233	…	…	…	…	…							
5	4	dd	24	F244	…	…	…	…	…							
6	5	ee	25	F255	…	…	…	…	…							
7	6	ff	26	F266	…	…	…	…	…							
8	7	gg	27	F277	…	…	…	…	…							
9	8	hh	28	F288	…	…	…	…	…							
10	9	ii	29	F299	…	…	…	…	…							
11	10	jj	30	F310	…	…	…	…	…							
12	…	…	…	…	…	…	…	…	…							

问题：学生aa在所有学年的平均分是多少?



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	ID	姓名	年龄	班级	学年	课程	成绩	…	…							
2	1	aa	21	F211	2020	kc-1	89	…	…							
3	2	bb	22	F222	2020	kc-1	85	…	…							
4	3	cc	23	F233	2020	kc-1	82	…	…							
5	4	dd	24	F244	2020	kc-1	81	…	…							
6	5	aa	21	F211	2021	kc-2	92									
7	6	ee	25	F255	2020	kc-1	84	…	…							
8	7	ff	26	F266	2021	kc-1	80	…	…							
9	8	gg	27	F277	2021	kc-1	80	…	…							
10	9	aa	21	F211	2022	kc-3	90									
11	10	hh	28	F288	2021	kc-1	80	…	…							
12	11	ii	29	F299	2021	kc-1	80	…	…							
13	12	jj	30	F310	2021	kc-1	80	…	…							
14	…	…	…	…	…	…	…	…	…							
15																

Web应用中前台页面的搜索其实都是转化成后台数据库的查询问题



什么是SQL

- **S**tructured **Q**uery **L**anguage，结构化查询语言，用于管理关系型数据库管理系统（RDBMS）
- SQL只是一种查询语言，**它不是数据库。** 要执行SQL查询，需要将它安装在数据库上例如Oracle，MySQL，MongoDB等
- 不同的数据库有不同的SQL方言

为什么SQL很重要?

- 是支撑实际业务的重要技术
- 可以在编程语言中嵌入SQL，实现数据处理

```
public class CrunchifyMySQLJDBCConnection {

    static Connection crunchifyConn = null;
    static Statement crunchifyStmt = null;
    static ResultSet crunchifyResultset = null;

    public static void main(String[] args) throws SQLException {
        System.out.println("-------- Crunchify's tutorial on Oracle JDBC Connectivity  ------");

        try {
            // Returns the Class object associated with the class
            Class.forName("oracle.jdbc.driver.OracleDriver");
        } catch (ClassNotFoundException exception) {
            System.out.println("Oracle Driver Class Not found Exception: " + exception.toString());
            return;
        }
```

用大白话讲，SQL就是用类似自然语言的查询语句方式从大量数据中返回想要的结果

常用的SQL命令

- **SELECT** - 从数据库中提取数据
- **UPDATE** - 更新数据库中的数据
- **DELETE** - 从数据库中删除数据
- **INSERT INTO** - 向数据库中插入新数据
- **CREATE DATABASE** - 创建新数据库
- **ALTER DATABASE** - 修改数据库
- **CREATE TABLE** - 创建新表
- **ALTER TABLE** - 变更（改变）数据库表
- **DROP TABLE** - 删除表
- **CREATE INDEX** - 创建索引（搜索键）
- **DROP INDEX** - 删除索引

SELECT语句

- 用于从数据库中选取数据
- SQL SELECT 语法

```
SELECT column1, column2, ...
FROM table_name;
```

```
SELECT * FROM table_name;
```

- 例：演示数据库表“website“如下

id	name	url	alexa	country
1	Google	https://www.google.cm/	1	USA
2	淘宝	https://www.taobao.com/	13	CN
3	菜鸟教程	http://www.runoob.com/	4689	CN
4	微博	http://weibo.com/	20	CN
5	Facebook	https://www.facebook.com/	3	USA

- 从"Websites" 表中选取 "name" 和 "country" 列

```
mysql> SELECT name,country FROM Websites;
+--------------+---------+
| name         | country |
+--------------+---------+
| Google       | USA     |
| 淘宝         | CN      |
| 菜鸟教程     | CN      |
| 微博         | CN      |
| Facebook     | USA     |
+--------------+---------+
```

SELECT语句

- **WHERE 子句**用于提取满足指定条件的记录
- 语法

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

- 例：从 "Websites" 表中选取国家为 "CN" 的所有网站

id	name	url	alexa	country
1	Google	https://www.google.cm/	1	USA
2	淘宝	https://www.taobao.com/	13	CN
3	菜鸟教程	http://www.runoob.com/	4689	CN
4	微博	http://weibo.com/	20	CN
5	Facebook	https://www.facebook.com/	3	USA

```
mysql> SELECT * FROM Websites WHERE country='CN';
+----+--------------+---------------------------+-------+---------+
| id | name         | url                       | alexa | country |
+----+--------------+---------------------------+-------+---------+
|  2 | 淘宝         | https://www.taobao.com/   |    13 | CN      |
|  3 | 菜鸟教程     | http://www.runoob.com/    |  4689 | CN      |
|  4 | 微博         | http://weibo.com/         |    20 | CN      |
+----+--------------+---------------------------+-------+---------+
3 rows in set (0.00 sec)
```

SELECT语句（更复杂的条件）

- **AND & OR 运算符**用于基于一个以上的条件对记录进行过滤
  - 如果第一个条件和第二个条件都成立，则 AND 运算符显示一条记录。
  - 如果第一个条件和第二个条件中只要有一个成立，则 OR 运算符显示一条记录。

例：结合AND & OR的查询

```
+----+--------------+---------------------------+-------+---------+
| id | name         | url                       | alexa | country |
+----+--------------+---------------------------+-------+---------+
| 1  | Google       | https://www.google.cm/    | 1     | USA     |
| 2  | 淘宝          | https://www.taobao.com/   | 13    | CN      |
| 3  | 菜鸟教程      | http://www.runoob.com/    | 4689  | CN      |
| 4  | 微博          | http://weibo.com/         | 20    | CN      |
| 5  | Facebook     | https://www.facebook.com/ | 3     | USA     |
+----+--------------+---------------------------+-------+---------+
```

1 2

```
mysql> SELECT * FROM Websites
    -> WHERE alexa > 15
    -> AND (country='CN' OR country='USA');
+----+--------------+-------------------------+-------+---------+
| id | name         | url                     | alexa | country |
+----+--------------+-------------------------+-------+---------+
|  3 | 菜鸟教程     | http://www.runoob.com/  |  4689 | CN      |
|  4 | 微博         | http://weibo.com/       |    20 | CN      |
+----+--------------+-------------------------+-------+---------+
2 rows in set (0.00 sec)
```

SELECT语句（更复杂的条件）

- **ORDER BY 关键字**用于对结果集进行排序
- SQL ORDER BY 语法

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

ASC：表示按升序排序
DESC：表示按降序排序

- 例：从 “Websites” 表中选取所有网站，并按照 “country”和“alexa” 列排序 （默认升序）

2 1

```
+----+--------------+---------------------------+-------+---------+
| id | name         | url                       | alexa | country |
+----+--------------+---------------------------+-------+---------+
| 1  | Google       | https://www.google.cm/    | 1     | USA     |
| 2  | 淘宝          | https://www.taobao.com/   | 13    | CN      |
| 3  | 菜鸟教程      | http://www.runoob.com/    | 4689  | CN      |
| 4  | 微博          | http://weibo.com/         | 20    | CN      |
| 5  | Facebook     | https://www.facebook.com/ | 3     | USA     |
+----+--------------+---------------------------+-------+---------+
```

```
mysql> SELECT * FROM Websites
    -> ORDER BY country,alexa;
+----+--------------+---------------------------+-------+---------+
| id | name         | url                       | alexa | country |
+----+--------------+---------------------------+-------+---------+
|  2 | 淘宝          | https://www.taobao.com/   |    13 | CN      |
|  4 | 微博          | http://weibo.com/         |    20 | CN      |
|  3 | 菜鸟教程      | http://www.runoob.com/    |  4689 | CN      |
|  1 | Google       | https://www.google.cm/    |     1 | USA     |
|  5 | Facebook     | https://www.facebook.com/ |     3 | USA     |
+----+--------------+---------------------------+-------+---------+
5 rows in set (0.00 sec)
```

结合函数构建更复杂的SQL查询

- **SQL GROUP BY语句：**用于结合聚合函数，根据一个或多个列对结果集进行分组
- 语法

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name;
```

- 例：统计网站的访问记录

```
mysql> SELECT * FROM access_log;
+-----+---------+-------+------------+
| aid | site_id | count | date       |
+-----+---------+-------+------------+
|   1 |       1 |    45 | 2016-05-10 |
|   2 |       3 |   100 | 2016-05-13 |
|   3 |       1 |   230 | 2016-05-14 |
|   4 |       2 |    10 | 2016-05-14 |
|   5 |       5 |   205 | 2016-05-14 |
|   6 |       4 |    13 | 2016-05-15 |
|   7 |       3 |   220 | 2016-05-15 |
|   8 |       5 |   545 | 2016-05-16 |
|   9 |       3 |   201 | 2016-05-17 |
+-----+---------+-------+------------+
```

```
mysql> SELECT site_id, SUM(access_log.count) AS nums
    -> FROM access_log GROUP BY site_id;
+---------+------+
| site_id | nums |
+---------+------+
|       1 |  275 |
|       2 |   10 |
|       3 |  521 |
|       4 |   13 |
|       5 |  750 |
+---------+------+
5 rows in set (0.00 sec)
```

结合函数构建更复杂的SQL查询

- HAVING子句可以筛选分组后的各组数据
- 语法

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1
HAVING condition;
```

- 例：查找总访问量大于 200 的网站

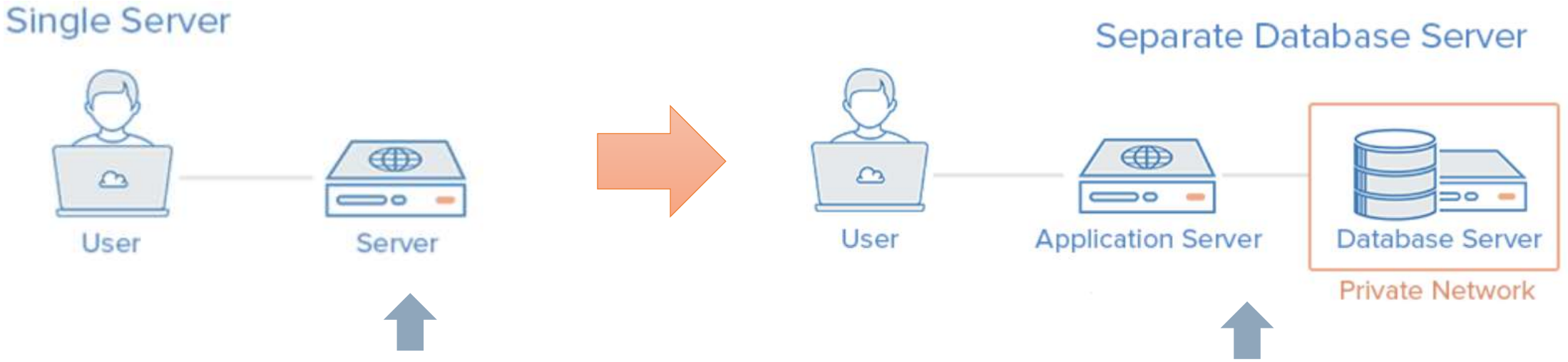
```
mysql> SELECT * FROM access_log;
+-----+---------+-------+------------+
| aid | site_id | count | date       |
+-----+---------+-------+------------+
|   1 |       1 |    45 | 2016-05-10 |
|   2 |       3 |   100 | 2016-05-13 |
|   3 |       1 |   230 | 2016-05-14 |
|   4 |       2 |    10 | 2016-05-14 |
|   5 |       5 |   205 | 2016-05-14 |
|   6 |       4 |    13 | 2016-05-15 |
|   7 |       3 |   220 | 2016-05-15 |
|   8 |       5 |   545 | 2016-05-16 |
|   9 |       3 |   201 | 2016-05-17 |
+-----+---------+-------+------------+
```

```
+----+--------------+---------------------------+-------+---------+
| id | name         | url                       | alexa | country |
+----+--------------+---------------------------+-------+---------+
| 1  | Google       | https://www.google.cm/    | 1     | USA     |
| 2  | 淘宝          | https://www.taobao.com/   | 13    | CN      |
| 3  | 菜鸟教程      | http://www.runoob.com/    | 4689  | CN      |
| 4  | 微博          | http://weibo.com/         | 20    | CN      |
| 5  | Facebook     | https://www.facebook.com/ | 3     | USA     |
| 7  | stackoverflow | http://stackoverflow.com/ |   0   | IND     |
+----+--------------+---------------------------+-------+---------+
```

```
mysql> SELECT Websites.name, Websites.url, SUM(access_log.count) AS nums FROM (access_log
    -> INNER JOIN Websites
    -> ON access_log.site_id=Websites.id)
    -> GROUP BY Websites.name
    -> HAVING SUM(access_log.count) > 200;
+--------------+---------------------------+------+
| name         | url                       | nums |
+--------------+---------------------------+------+
| Facebook     | https://www.facebook.com/ |  750 |
| Google       | https://www.google.cm/    |  275 |
| 菜鸟教程 | http://www.runoob.com/    |  521 |
+--------------+---------------------------+------+
3 rows in set (0.00 sec)
```

典型的系统应用架构

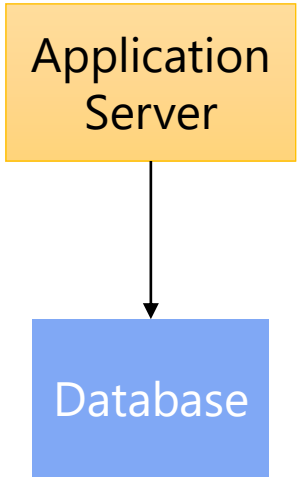
- 从单个服务器到数据服务器分离



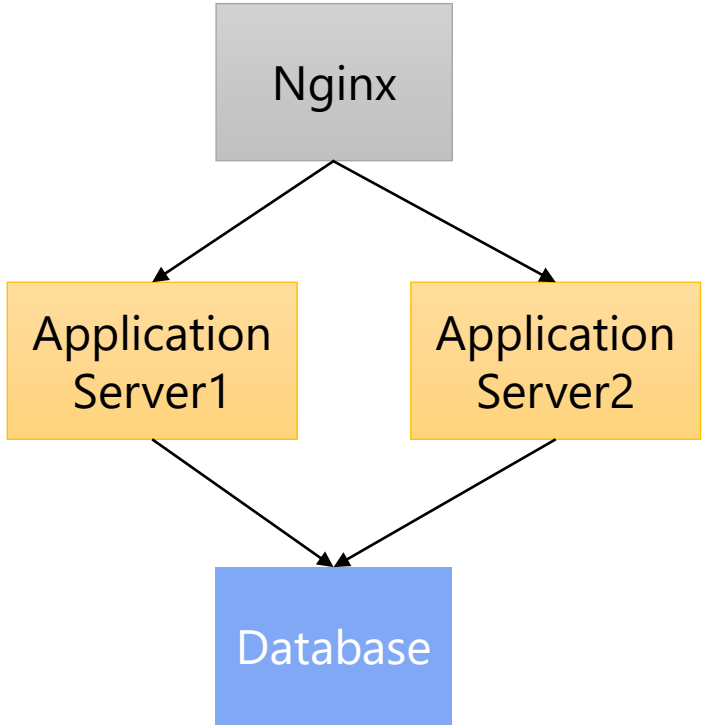
早期，应用程序、数据库、文件等**所有资源都在一台服务器上**，通常只要使用 Linux PHP MySQL Apache 就可以完成整个项目部署，然后再买个域名，就能运行一个网站了

应用和数据分离。**应用服务器负责处理大量的业务逻辑**，通常需要强大的CPU，**数据库服务器需要快速的进行磁盘检索和数据缓存**，因此需要更快的硬盘和更大的内存

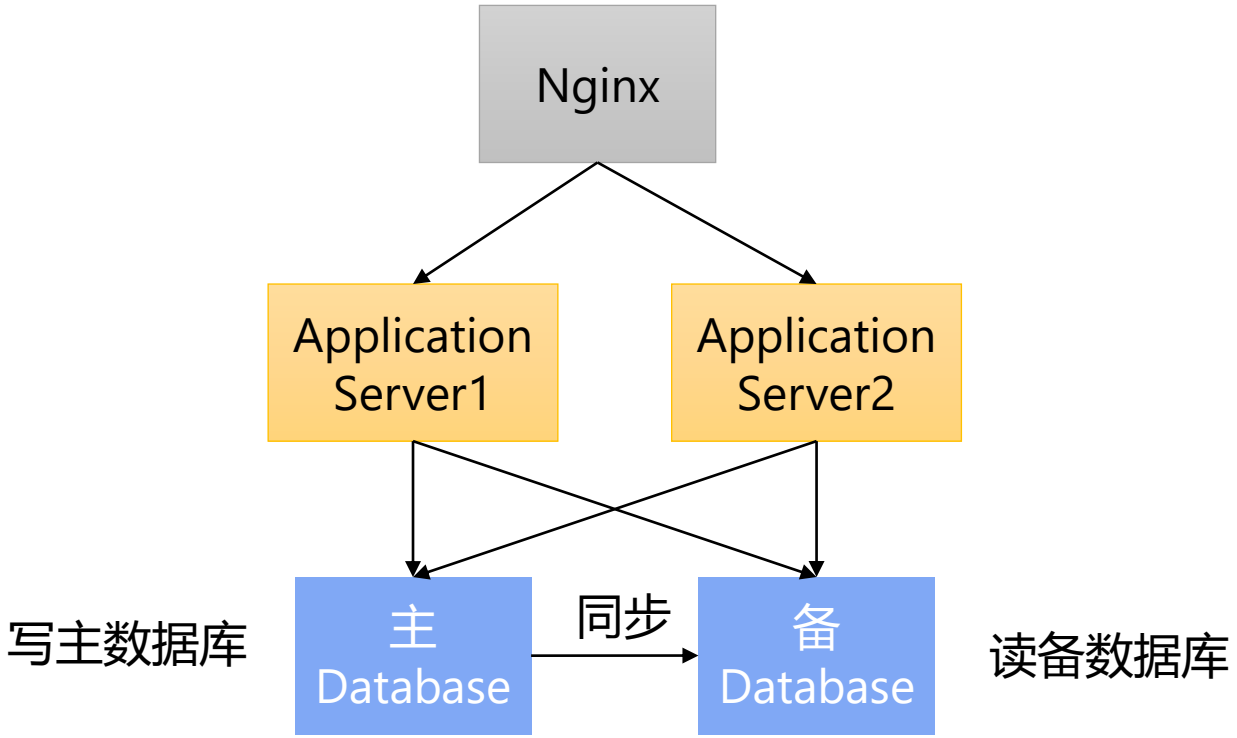
- 阶段1：企业刚发展，一个应用服务器配置一个关系型数据库，每次读写数据库



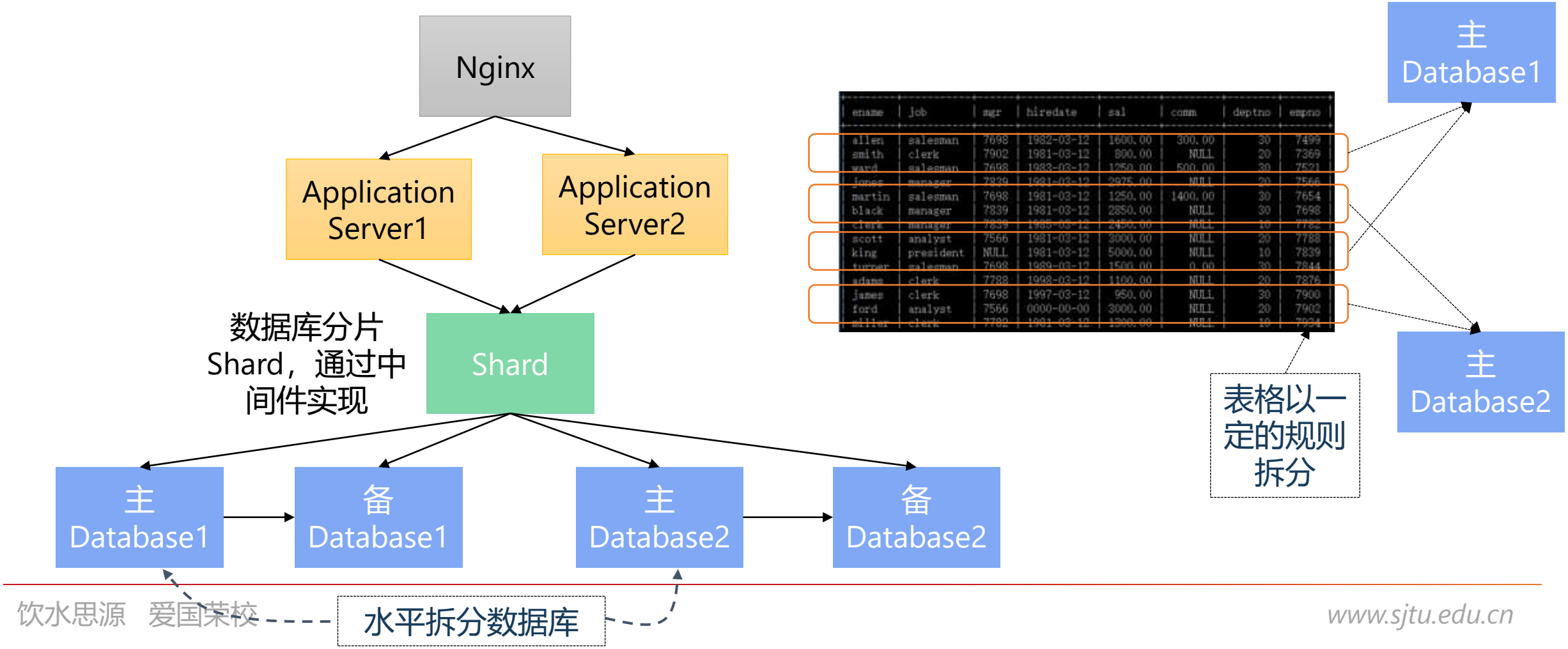
- 阶段2：企业的业务取得一定的发展，一台应用服务器无法承担所有的上游流量，但数据库**仍然不会成为瓶颈**，这时增加应用服务器并在流量入口使用Nginx做均衡负载



- 阶段3：随着企业的业务规模继续扩大，对于数据库的读写出现了瓶颈，这时需要**对数据库做读写分离**，解决数据库的性能问题



- 阶段4：企业的业务持续做大，读写分离的数据库压力也越来越大，需要拆分数据库，**做分库分表**



基于关系型数据库的存储架构演进

关系型数据库(比如MySQL)集群存在的问题

- 管理和配置复杂
- 数据库备份和复制不方便
- 难以动态扩容
- 数据迁移困难

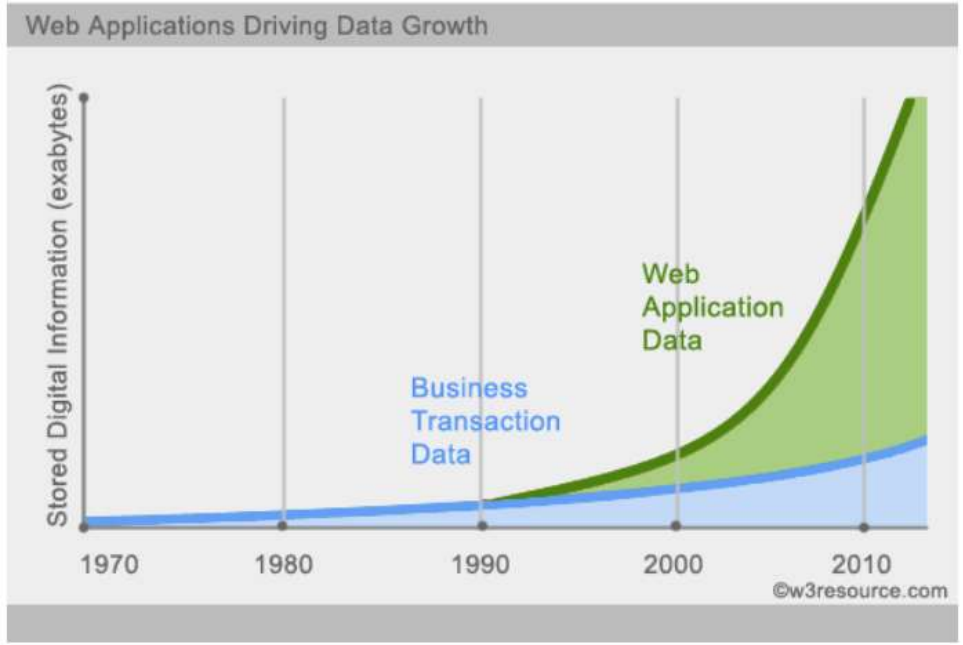
NoSQL的兴起

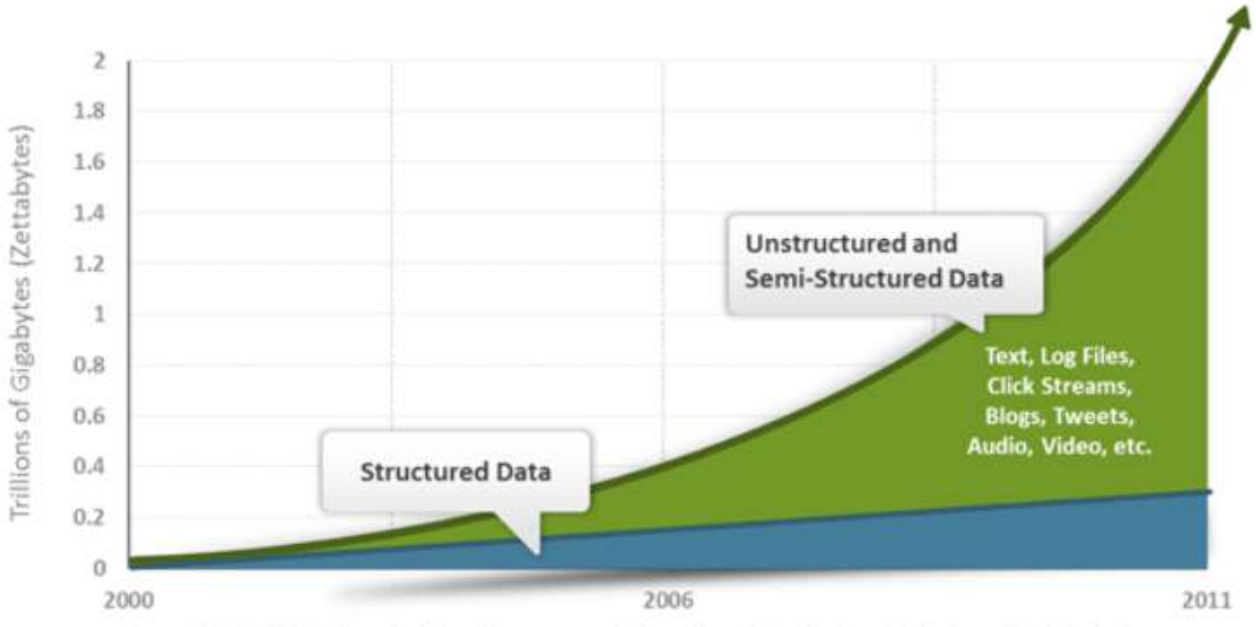
- **关系数据库的关键特性包括完善的事务机制和高效的查询机制。但是，关系数据库引以为傲的两个关键特性，到了Web2.0时代却成了鸡肋，主要表现在以下几个方面：**
  - Web2.0网站系统通常不要求严格的数据库事务
  - Web2.0并不要求严格的读写实时性
  - Web2.0通常不包含大量复杂的SQL查询（去结构化，存储空间换取更好的查询性能）



NoSQL的兴起

- 现在我们通过第三方平台（如Google,Facebook等）可以很容易地访问和抓取数据。用户的个人信息、社交网络、地理位置、用户生成的数据和用户操作日志已经成倍的增加。我们如果要对这些用户数据进行挖掘，那SQL数据库已经不适合这些应用了，NoSQL 数据库的发展却能很好的处理这些大的数据。





NoSQL的概念

- NoSQL一词最早出现于1998年，是Carlo Strozzi开发的一个轻量、开源、不提供SQL功能的关系数据库。
- 2009年，Last.fm的Johan Oskarsson发起了一次关于分布式开源数据库的讨论，来自Rackspace的Eric Evans再次提出了NoSQL的概念，这时的NoSQL主要指非关系型、分布式、***不提供ACID的数据库设计模式。***
- 2009年在亚特兰大举行的"no:sql(east)"讨论会是一个里程碑，其口号是"select fun, profit from real_world where relational=false;"。因此，对NoSQL最普遍的解释是"非关联型的"，强调Key-Value Stores和文档数据库的优点，而不是单纯的反对RDBMS。



概念演变 →

Not only SQL

最初表示“反SQL”运动
用新型的非关系数据库取代关系数据库

现在表示关系和非关系型数据库各有优缺点
彼此都无法互相取代

关系型数据库的事务遵循ACID规则

- **A (Atomicity) 原子性**
  - 事务里的所有操作要么全部做完，要么都不做
- **C (Consistency) 一致性**
  - 事务的运行不会改变数据库原本的一致性约束
- **I (Isolation) 独立性**
  - 并发的事务之间不会互相影响，如果一个事务要访问的数据正在被另外一个事务修改，只要另外一个事务未提交，它所访问的数据就不受未提交事务的影响
- **D (Durability) 持久性**
  - 一旦事务提交，则其所做的修改就会永久保存到数据库中。此时即使系统崩溃，修改的数据也不会丢失

事务步骤	商品库存	是否一致
初始状态	10	✅
用户下单购买15个（检查约束）	-5（拒绝操作）	❌（违反约束）
事务回滚	10（保持不变）	✅

事务步骤	账户A	账户B	总金额	是否一致
初始状态	1000	1000	2000	✅
A转100给B（执行中）	900	1000	1900	❌（暂时不一致）
事务提交后	900	1100	2000	✅

事务：一系列数据操作或指令组成的一个不可分割的逻辑单元

总结来讲：

- 关系数据库和NoSQL数据库各有优缺点，彼此无法取代，两者应该互补
- **关系数据库应用场景**：电信、银行等领域的关键业务系统，需要保证强事务一致性
- **NoSQL数据库应用场景**：互联网企业、传统企业的非关键业务（比如数据分析）

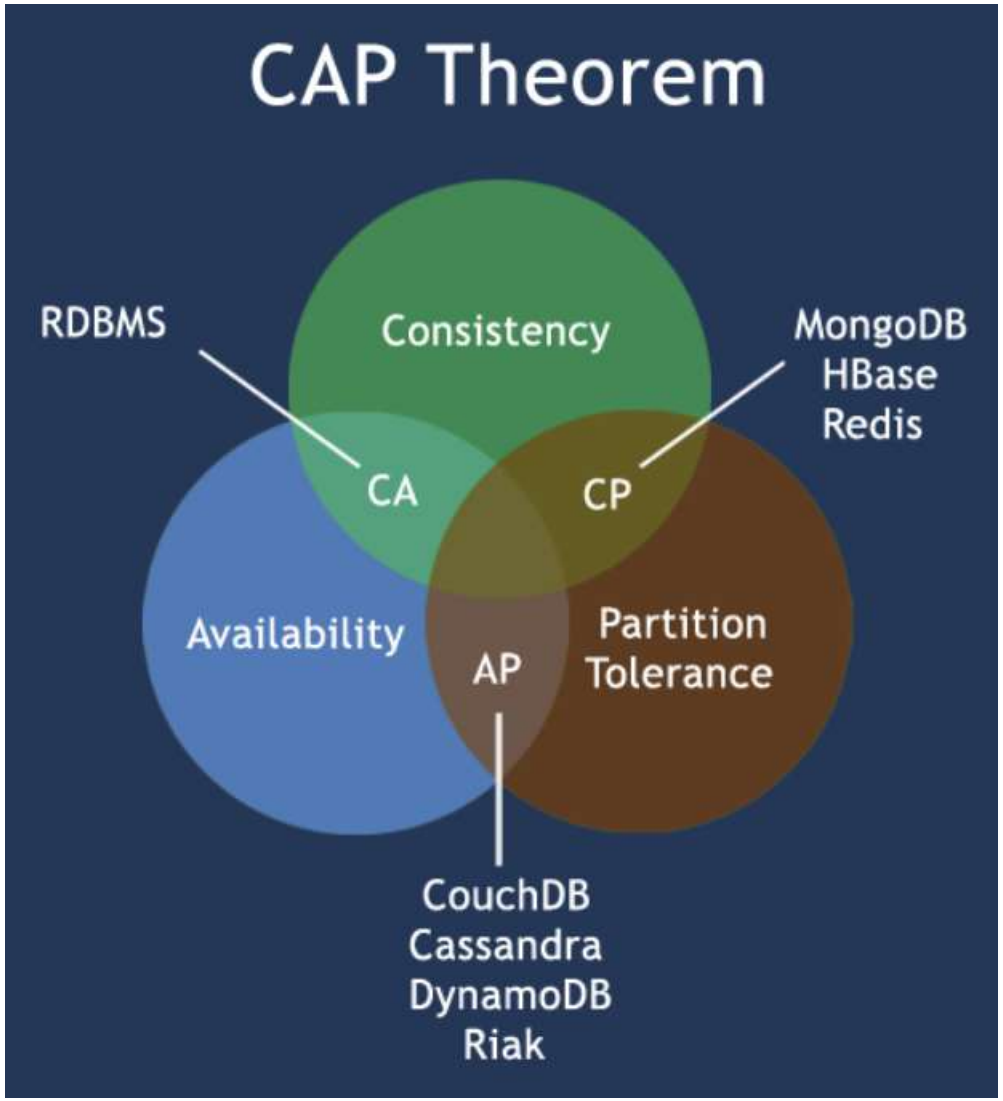
- **采用混合架构**
  - 案例：亚马逊公司使用不同类型的数据库来支撑它的电子商务应用
    - 对于“购物篮”这种临时性数据，采用键值存储会更加高效
    - 当前的产品和订单信息则适合存放在关系数据库中
    - 大量的历史订单信息则适合保存在类似MongoDB的文档数据库中

CAP定理

- 在计算机科学中, **CAP定理**（CAP theorem），又被称作 布鲁尔定理（Brewer's theorem），它指出对于一个分布式计算系统来说，不可能同时满足以下三点:
  - **一致性(C**onsistency)：所有节点在同一时间具有相同的数据
  - **可用性(A**vailability)：保证每个请求不管成功或者失败都有响应
  - **分区容错性(P**artition tolerance)：系统中任意信息的丢失或失败不会影响系统的继续运作

NoSQL

- CAP理论的核心是：一个分布式系统不可能同时很好的满足一致性，可用性和分区容错性这三个需求，**最多只能同时较好的满足两个。**
- 因此，根据 CAP 原理将 NoSQL 数据库分成了满足 **CA** 原则、满足 **CP** 原则和满足 **AP** 原则三大类：
  - **CA**： 满足一致性、可用性的系统。通常在可扩展性上不太强大
    - 传统RDBMS
  - **CP**： 满足一致性、分区容忍性的系统。通常性能不是特别高
    - 当出现网络分区的情况时，受影响的服务需要等待数据一致，因此在等待期间就无法对外提供服务
  - **AP**： 满足可用性、分区容忍性的系统。通常可能对一致性要求低一些
    - 允许系统返回不一致的数据



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Thank You

